

REPORT

AppSec Overflow 2026

How AI broke the math of application security

Table of Contents

Executive summary.....	3
Introduction.....	4
The software threat landscape: Attackers are reaching real vulnerabilities.....	5
→ Attack volume.....	6
→ Attack breakdowns.....	7
Software vulnerabilities: The attack surface is expanding faster than defenders can keep up.....	10
→ Vulnerability volume.....	10
→ Noteworthy high-prevalence CVEs.....	12
Automation chaos: AI is making the problem worse as it's making it better	13
Vulnerability prioritization: Defenders face a prioritization crisis.....	14
→ Application vulnerability categories.....	14
→ Prioritization by vulnerability severity and exploitability: Imperfect metrics.....	17
Retooling AppSec for the AI age.....	19
Contrast was built for the age of AI.....	19

Executive summary

AppSec math is broken, and in 2026, AI is a massive contributing factor. Application and API security has always been an asymmetric fight. Attackers need to find just one way in, while defenders need to seal every gap to successfully keep them out. Now, however, AI is poised to overflow the strained capacity of AppSec teams. It accelerates exploitation, floods pipelines with new vulnerabilities, and offers defenders tools whose findings often contradict one another, forcing teams to reconcile the noise before they can act. Teams are working harder to keep up, only to find themselves falling further behind.

The findings in this report draw from [Contrast's runtime security platform](#), which collects telemetry from within hundreds of thousands of applications and APIs at industry leading brands across a broad range of sectors in production environments worldwide. Because Contrast operates from inside the application stack rather than at the perimeter, it captures the full context of how applications behave under attack, providing a unique perspective on the state of application security. Key findings include:

- **Attackers are reaching real application vulnerabilities every day.** The average application faces 42 confirmed, viable exploit attempts every month, and attackers touch running applications and APIs once every 4 minutes, on average. Third-party data show these exploits are coming faster as well, with time-to-exploit shifting from years in 2018 to just hours in 2026.
- **The attack surface is expanding faster than defenders can keep up.** The average app carries 22 serious vulnerabilities, yet teams fix just 3.4 vulnerabilities per app per month. Patching vulnerabilities takes months: critical vulnerabilities in custom code take an average of 92 days to remediate, and 54% of observed third-party vulnerabilities are more than a year old. Teams dealing with this already-underwater baseline are set to drown under the weight of AI-powered vulnerability discovery.
- **AI is making this worse as much as it's making it better.** Throwing AI at this problem isn't the silver bullet it's often made out to be for security teams. Contrast's research shows that AI scanners don't agree on what they've found. Three scanners analyzing the same codebase agreed on only 5% of findings, and one AI scanner reproduced only 17% of its own findings across repeated runs. Add in the waves of new AI-generated findings, and it's clear that AI isn't going to dig us out of this hole any time soon; for now, it's making the hole a lot deeper.
- **Defenders face a prioritization crisis.** The metrics used by AppSec teams to prioritize vulnerabilities for remediation provide an incomplete and sometimes misleading picture of risk. CVSS, EPSS and KEV are valuable, but also sometimes point in conflicting directions and don't provide the surgical focus that defenders need in the age of AI.

It's time to retool AppSec for the age of AI. The way forward is not to blindly apply patches and controls based on imperfect predictors, but to act confidently based on objective runtime observations.

Runtime application security: A ground-truth view of risk

The data in this report comes from inside applications and APIs themselves. Contrast's runtime application security platform embeds a lightweight sensor into each monitored application and API, providing continuous visibility into how that software actually behaves in production. As requests flow through the stack, the sensor observes control flow, data flow, library invocations and backend interactions such as SQL queries, file access and external service calls.

This inside-out perspective answers questions that perimeter and endpoint tools cannot: Is an application vulnerability reachable by user input? Is it triggerable along actual execution paths? Which library versions are actually loaded and exposed at runtime? The vantage point lets us track not just which vulnerabilities exist in code, but which ones are present and exploitable in live production environments.

The findings that follow are drawn from anonymized telemetry spanning thousands of applications and trillions of security-critical observations each day.

Introduction

Application security has run on the same workflow for two decades: find vulnerabilities, prioritize them, and patch them before attackers can exploit them. That model is now collapsing, and AI is the driving force.

The find-and-fix cycle rests on two assumptions:

1. Defenders can discover vulnerabilities at a pace human teams can absorb and identify the ones that matter.
2. Defenders can patch faster than attackers can weaponize a vulnerability.

AI has broken both of these assumptions at once. When it comes to uncovering vulnerabilities in applications and APIs, AI is capable of generating new vulnerability findings at a rate no AppSec team can keep up with. At the same time, attackers are using AI to produce working exploits and find vulnerable systems before patches finish deploying. The same technology is destabilizing the workflow from both directions at machine speed.

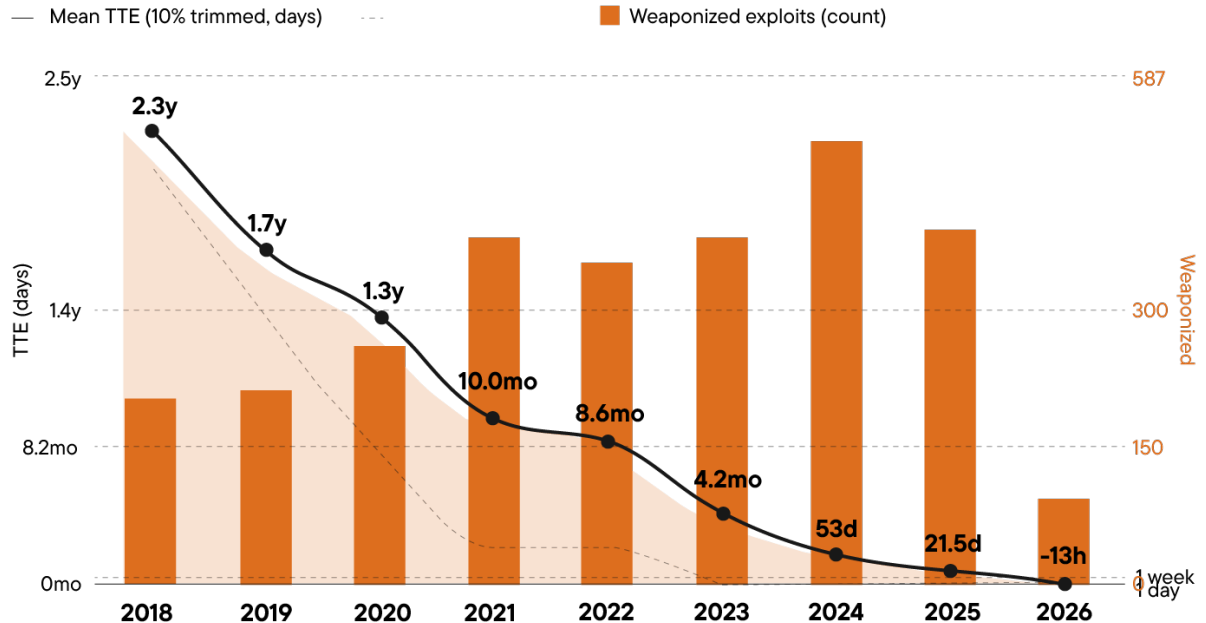
The discovery side of this shift came sharply into focus in April 2026 when Anthropic introduced Claude Mythos Preview. The Mythos AI model demonstrates an alarming capability for autonomous vulnerability research. Anthropic has not disclosed the time, accuracy and human effort that was required for their analysis, making it difficult for security leaders to make real risk assessments today. Regardless, each critical flaw identified in a widely used library, kernel or browser engine eventually flows into the public CVE feed, where every enterprise running that software inherits a new finding to triage and remediate.

In early testing, Mythos is reported to have discovered thousands of previously unknown zero-day vulnerabilities across multiple major operating systems and web browsers, including a flaw in OpenBSD that had survived 27 years of human review. The security community has a name for what comes next: "The Vulnpocalypse,"¹ a term coined to describe the structural collapse that occurs when AI-accelerated vulnerability discovery permanently outpaces the human capacity to remediate.

The strain has brought real-world impacts. In March 2026, HackerOne paused new submissions to the Internet Bug Bounty (IBB) program,² the longest-running and most heavily funded crowdsourced vulnerability program in open source. AI-assisted discovery had expanded coverage and speed to the point where the volume of incoming reports outstripped the capacity of small maintainer teams to validate and fix them. Node.js, which relied on IBB funding, paused its own program shortly after.³ HackerOne was clear about their reasons: the balance between findings and remediation capacity has substantively shifted, and the bottleneck has moved from finding bugs to fixing them.

The offensive side is moving in lockstep. Zero Day Clock, which aggregates exploit signals from CISA's Known Exploited Vulnerabilities catalog, ExploitDB, Metasploit, and other public sources across more than 83,000 CVEs, tracks the collapse of time-to-exploit. In 2018, the average time from CVE disclosure to first observed exploit was more than two years. By 2021, it was less than one year. In 2025, the majority of exploited vulnerabilities were weaponized within 3 weeks, and the number has shrunk to just hours in 2026. The trajectory fits an exponential decay curve, and AI is the engine helping to drive it. Independent research from VulnCheck corroborates the trend, finding that 29 percent of exploited CVEs in 2025 were weaponized on or before the day their CVE was published.⁴

Figure 1: The rapidly shrinking time from vulnerability release to exploitation



Source: Zero Day Clock

This is not a gap that better scanners, more analysts or faster patch cycles can close. The structural assumption underneath traditional AppSec, that defenders can outrun the find-and-fix clock, no longer holds in the age of AI. Security leaders who choose to continue investing in an AppSec model built for human-speed attackers and pre-AI vulnerability volumes will end up watching the exposure gap widen month over month. To change this dynamic, organizations must pivot to a defensive posture designed for the AI era, one that treats production application behavior as the source of truth and protects running code in real time, regardless of whether the underlying vulnerability has been discovered, disclosed, cataloged, prioritized or patched.

The remainder of this report examines what that pivot looks like through the lens of Contrast's runtime telemetry: how application-layer attacks are unfolding right now, how the vulnerability landscape is evolving under AI pressure, and where the remediation gap is widening fastest.

The following sections provide a breakdown of recent attack activity observed by Contrast. These insights are designed to help SOC teams prioritize detection and response efforts where they will have the greatest impact.

The software threat landscape: Attackers are reaching real vulnerabilities

Adversaries targeting today's applications operate at machine speed. They run automated, AI-powered tooling that continuously probes for weaknesses at scale, and move quickly when they find one. The data Contrast collects from thousands of production applications and APIs gives us an unusually clear window into this activity and what it looks like hitting real applications, in real environments, every day.

Of course not all "attacks" are equally important. Too often, alert queues are filled with alerts that turn out to be false positives, automated probes for potential weaknesses, or spray-and-pray attacks targeting non-existent vulnerabilities. Contrast's unique perspective allows us to focus attention on the truly viable attacks. These events are the real signal buried in the noise, the threats that merit analyst attention and response.

The data shows that application-layer attacks are not occasional events. They are background noise that never stops, punctuated by confirmed daily exploitation attempts that demand immediate attention. The challenge for SOC teams is that most of this activity is invisible to the tools they rely on most. Network and endpoint signals don't reach deep enough into the application stack to distinguish a genuine application exploit from routine traffic.

The sections below break down what Contrast observed across its monitored application and API population over the last 12 months, including how frequently attacks occur, which techniques dominate, and how attack patterns shift across industries. Together these data give security leaders the context they need to make smarter decisions about where to focus detection and response resources.

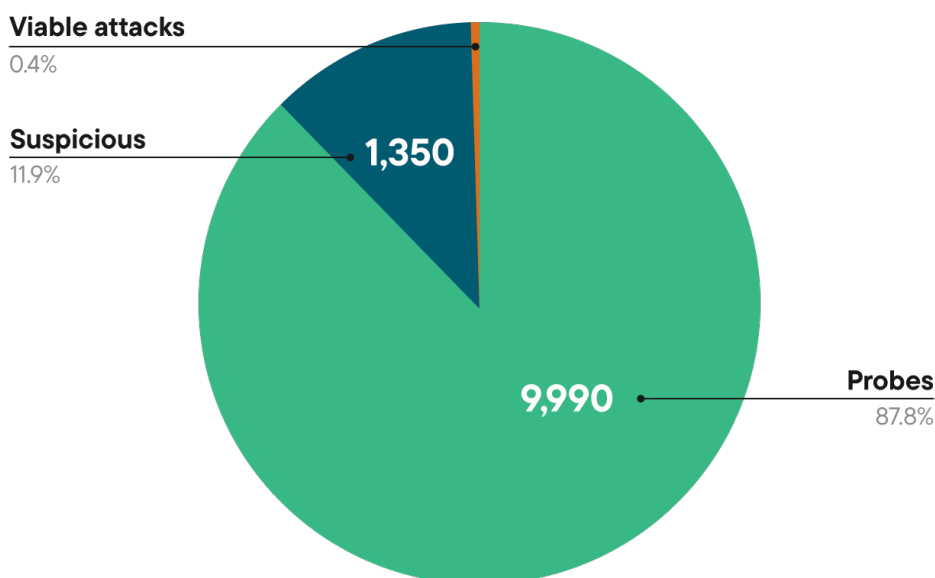
Attack volume

Contrast's telemetry puts hard numbers on a threat that many organizations still underestimate. Across thousands of production applications and APIs, the average application absorbs 11,382 attacks per month, roughly one every four minutes.

To understand what lies behind that headline number, it helps to break attacks into three categories based on their nature and potential impact.

Attackers touch running applications and APIs every 4 minutes

Figure 2: Average number of attacks per application per month, by type



The average application faces 42 viable attacks/month

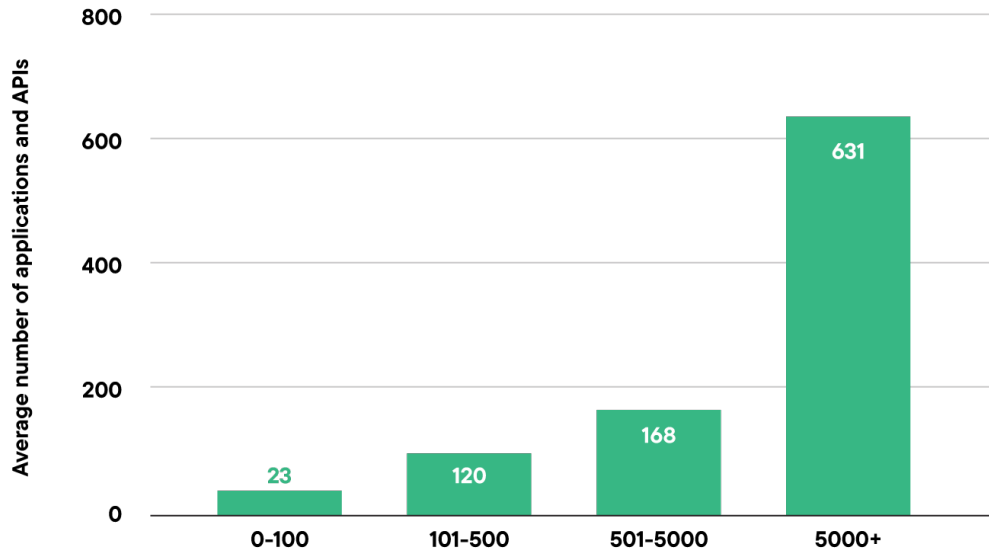
Not all attack activity carries the same weight, and treating it as equivalent is one of the ways security teams get buried. The 11,000-plus monthly attacks against a typical application break down into three distinct categories, each representing a different level of threat.

- Probes make up the bulk of the volume. These are automated reconnaissance attempts, scanning for known weaknesses, cataloging services, and mapping attack surfaces. They rarely cause direct harm, but they signal active interest and often precede more targeted follow-on activity.
- Suspicious attacks go further. These show clear indicators of malicious intent: exploit payloads, evasion techniques, attempts to manipulate application behavior. They haven't been confirmed as successful exploits, but they represent credible threats that warrant attention and investigation.
- Viable attacks are where the real danger lives. These are exploitation attempts that Contrast has confirmed actually reached and triggered a real vulnerability in the running application. There are no false positives here; runtime observation proves the vulnerability wasn't just probed; it was activated. The average application faces 42 of these per month.

For a large enterprise running hundreds of applications, that number scales fast. Below is a breakdown of the average number of applications and APIs per organization, by employee count.

An organization with hundreds of applications is absorbing tens of thousands of confirmed exploitation attempts every month, each one a potential starting point for something far worse.

Figure 3: Average number of applications and APIs per organization, by employee count

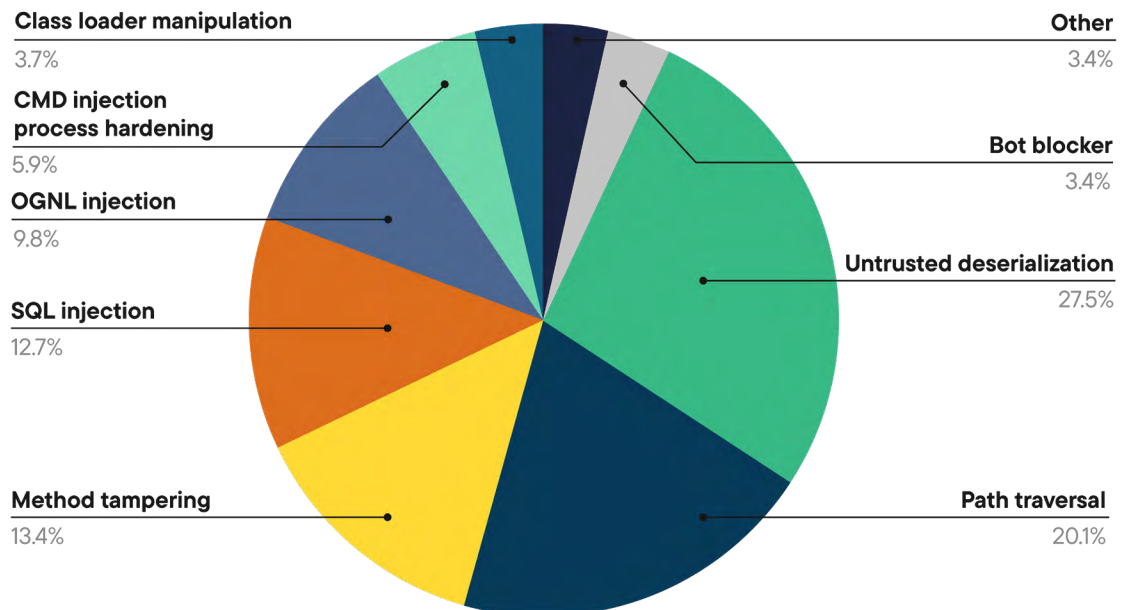


Attack breakdowns

Volume tells us how much pressure applications are under; the breakdown by attack type tells us something more useful: what adversaries are actually trying to do, which techniques they're betting on, and where the real exposure lies. The following data breaks down viable attack activity by technique and by industry, giving security teams the context they need to move beyond generic defenses and focus on the threats most relevant to their environment.

The viable attack data reveals a threat landscape that is both diverse and technically sophisticated. Untrusted deserialization leads the field, a technique that has proven stubbornly persistent because it is difficult to detect

Figure 4: Top types of viable application attacks



without runtime visibility inside the application or API. Path traversal follows, a reminder that fundamental input validation failures remain as exploitable as ever despite being well understood and well documented. Method tampering ranks prominently as well, exploiting misconfigured HTTP authentication and access controls that appear across virtually every technology stack.

When broken down by industry, attack patterns shift in ways that reflect each sector's architecture, exposure profile, and the adversaries targeting it.

Figure 5: Top five viable attack techniques by industry vertical

	Finance	Healthcare	Manufacturing	Services	Tech
#1	SQL injection	OGNL injection	Method tampering	Untrusted deserialization	Class loader manipulation
#2	CMD injection process hardening	CMD injection process hardening	Path traversal	OGNL injection	CMD injection process hardening
#3	Path traversal	Bot blocker	OGNL injection	Class loader manipulation	Path traversal
#4	Untrusted deserialization	Path traversal	SSJS injection	Method tampering	Method tampering
#5	OGNL injection	SQL injection	SQL injection	SQL injection	SQL injection

Several techniques appear broadly across sectors. SQL injection, for example, appears in the top five for every vertical in this year's data, a reminder that fundamental input validation failures remain pervasive across all industries and technology stacks. OGNL injection similarly spans multiple sectors, reflecting the continued prevalence of Java-based enterprise frameworks as an attack surface.

Other patterns are more sector-specific. Untrusted deserialization leads the viable attack data for the services vertical, consistent with the heavy use of Java middleware and API-driven architectures in that sector. Class loader manipulation appears prominently in the technology vertical, a more sophisticated technique that targets the Java class loading mechanism itself and is consistent with adversaries pursuing deeper persistence in software-rich environments. Manufacturing continues to show a distinct profile driven by the constraints of operational technology environments: slower patching cycles, heavy reliance on third-party software components, and aging frameworks that may remain in production long after security support has lapsed.

Technique spotlight: Untrusted deserialization

What it is:

Untrusted deserialization attacks exploit vulnerabilities in code designed to reconstruct structured data objects, including Java objects, .NET types and PHP serialization. When applications reconstruct these objects without strict validation or type constraints, attackers can inject malicious payloads that execute within the application's own process.

How attackers exploit it:

Attackers craft malicious serialized objects that, when processed by the application, can trigger remote code execution, escalate privileges, or disrupt service. Detection is particularly difficult because the payload arrives as what appears to be legitimate structured data, opaque to network and endpoint tools that have no visibility into what the application does with it.

How to defend:

- ❑ Avoid deserialization of untrusted, user-controlled input wherever possible.
- ❑ Use safer serialization formats that do not support complex object references or embedded type information.
- ❑ Keep libraries and frameworks updated. Many deserialization vulnerabilities live in third-party components, and ensuring serialization-related dependencies are up to date removes a significant share of exposure.
- ❑ Enable runtime protections that observe deserialization behavior in context and can detect and block anomalous object reconstruction before it causes harm.

Technique spotlight: Path traversal

What it is:

Path traversal attacks exploit insufficient validation of file path inputs, allowing attackers to navigate outside the intended directory structure and access files the application was never meant to expose, including configuration files, credential stores and other sensitive data.

How attackers exploit it:

By injecting sequences such as `../` into URLs or parameters that reference files or directories, attackers can step up through the directory tree and reach files outside the application's intended scope. Automated scanning tools make it trivial to test for this weakness at scale across large numbers of applications simultaneously.

How to defend:

- ❑ Validate and sanitize all file path inputs, rejecting or neutralizing traversal sequences before they reach file system calls.
- ❑ Use allow-lists rather than block-lists, defining exactly which files or directories an application should be able to access and rejecting anything outside that scope.
- ❑ Apply the principle of least privilege to application file system access, limiting what is reachable even if a traversal attempt succeeds.
- ❑ Runtime instrumentation can observe the actual file system calls an application makes, detecting traversal attempts that evade input-layer controls.

Takeaway

Tabulating the volume of application-layer attacks shows that they are relentless, technically diverse, and shaped by the specific environments adversaries are targeting. The data makes clear that no single defense covers the full range of techniques in play. Organizations that understand their own threat profile, and have visibility into what is actually happening inside those applications and APIs are in a fundamentally better position than those relying on volume-based alerting and perimeter controls alone.

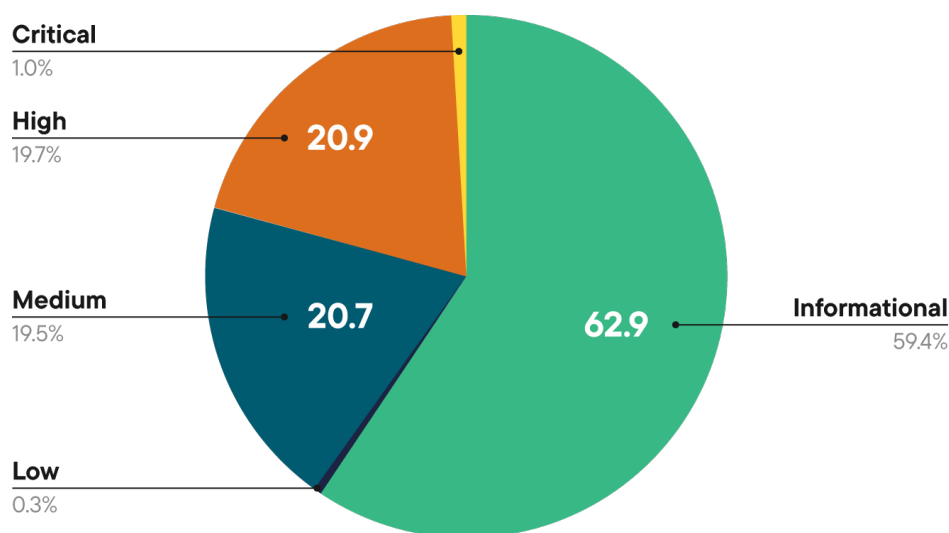
Software vulnerabilities: The attack surface is expanding faster than defenders can keep up

The attack activity documented in the previous section does not happen in a vacuum. Every successful application-layer exploit begins with an unpatched vulnerability, either in an organization's own code or in the third-party libraries and frameworks it depends on. The question is never whether vulnerabilities exist. The question is which ones matter, how quickly defenders can close them, and whether the tools and practices organizations rely on to answer those questions are actually up to the job.

Vulnerability volume

The volume of vulnerability findings in any production environment has always exceeded the capacity of the teams responsible for remediating them. Contrast's telemetry reveals the precise shape of that gap by applying architectural, threat, and business context to understand the real risk associated with issues and the 2-5% that truly matter.

Figure 6: Average vulnerability findings per application, by severity



Looking at code developed in-house, the average application monitored by Contrast carries around 106 vulnerability findings. At first glance, that number might seem alarming. In practice, the picture is more nuanced. The majority of findings, about 60%, are rated Informational, representing observations that flag potential hygiene issues but carry little real-world exploit risk in a production context.

The concern lies in what's underneath: an average of 22 serious vulnerabilities, rated high or critical per application. High and critical severity findings represent the combination of exploitability and potential impact that attackers actively seek out. In an environment where the time from vulnerability disclosure to active exploitation can be measured in hours, 22 serious vulnerabilities per application represents significant exposure.

The average
application carries
106
vulnerability findings

The average
application carries
22
serious vulnerability findings

Remediation throughput tells the other side of the story. Development and AppSec teams are closing an average of 3.4 vulnerabilities per application per month. Focusing for the moment on only critical vulnerabilities, the mean time to remediate a critical application vulnerability is 92 days.

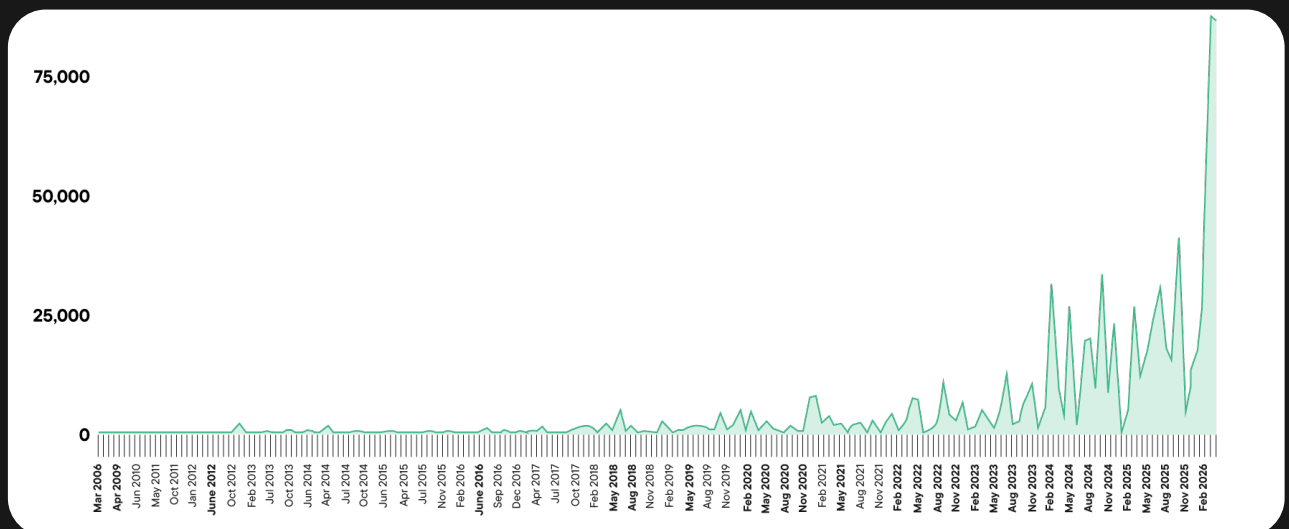
Patching a production system is not a simple operation: it requires change management, dependency analysis, regression testing, and coordination across development, operations, and security teams. Maintenance windows are limited, and competing priorities are all around. The result is too few vulnerabilities are being addressed, over too long a time, giving AI-powered attackers the runway they need to identify and exploit them.

92 days
Average time to remediate
critical application-layer
vulnerabilities

3.4 vulns/app/month
Average number of
vulnerabilities remediated per
month per application

Vulnerabilities in third-party code are not addressed any more quickly. Charting observed vulnerability instances by the date of CVE publication reveals the long tail of vulnerability remediation.

Figure 7: Observed AppSec CVE instances by publication month



54% — more than half — of CVE instances Contrast observed in production applications are from CVEs published more than a year ago. A meaningful share goes back several years. The world's collective patching performance, as reflected in real production environments monitored by Contrast, is not keeping pace with the ongoing accumulation of known vulnerabilities.

54%
of AppSec CVEs
observed in production
are more than 1 year old

This matters for several reasons. First, older vulnerabilities are better-documented and more reliably exploitable. Attackers who invest time in developing exploit capability for a given CVE want that investment to pay off across as many targets as possible for as long as possible. A CVE that has been public for two years has had two years of exploit refinement and automation. Second, exploits for older vulnerabilities are more likely to have been incorporated into commodity attacker tooling, lowering the skill threshold for attackers who want to use them. Third, the presence of years-old CVEs in production environments confirms that patching cycles are not systematically addressing the backlog.

Noteworthy high-prevalence CVEs

The following CVEs exemplify different facets of the vulnerability persistence problem. Each appears with meaningful frequency across Contrast's monitored customer base, and each illustrates a different dimension of why old and serious vulnerabilities continue to represent live risk.

CVE-2022-22965 - Spring4Shell

Disclosed in 2022, this RCE vulnerability in the Spring Framework remains one of the most widely observed CVEs in Contrast's dataset. Its continued presence four years after disclosure is a clear indicator that a meaningful population of internet-facing applications are still running unpatched Spring components, whether through neglect or supply chain blind spots.

CVE-2023-46604 - Apache ActiveMQ Deserialization RCE

This deserialization RCE vulnerability has been adopted as an initial access vector by multiple ransomware families, including HelloKitty, LockBit and TellYouThePass, as well as the Kinsing cryptomining botnet. Any unpatched ActiveMQ instance in a production environment should be treated as actively at risk.

CVE-2025-24813 - Apache Tomcat Path Equivalence RCE

Disclosed in March 2025, this path equivalence flaw allows an unauthenticated attacker to upload a malicious serialized file and trigger remote code execution via deserialization. Attackers began mass scanning for vulnerable instances within 30 hours of disclosure and continue to do so today.

CVE-2021-44228 - (Log4Shell) and CVE-2021-45046

Log4Shell triggered one of the largest mass-remediation efforts in enterprise security history. Its continued presence in Contrast's telemetry in 2026, more than four years after disclosure, reflects the supply chain complexity of modern Java applications, where Log4j can be bundled inside third-party dependencies without the application team's knowledge.

Takeaway

The picture that emerges from this section is not a patching problem. It is a structural one. The combination of high baseline volume, aging backlogs, constrained remediation capacity and an incoming wave of AI-discovered vulnerabilities means that most organizations are not falling behind because of poor execution; they are falling behind because the math does not work in their favor.

For security leaders, the implication is clear: a strategy built primarily on patching will never close the gap. The goal is to ensure that the vulnerabilities attackers are most likely to reach and to exploit are the ones getting addressed first, and that production applications are defended even when remediation is still in progress. For that we need more effective prioritization.

Automation chaos: AI is making the problem worse as it's making it better

Faced with large queues of findings, the natural inclination for many security teams in 2026 is to apply AI to the problem. There's a move within the AppSec community to fight fire with fire: leveraging AI to help defenders find, prioritize and fix vulnerabilities faster. Unfortunately, these tools are often contributing more chaos and cost than they resolve. In a recent research report titled "[The Hidden Cost of AI Security Scanners](#)" Contrast researchers found that:

- ❖ The API cost is the smallest line item when it comes to managing risk of application vulnerabilities. In one test, the token costs to scan an application with 2 million lines of code was around \$315, the costs to triage those findings was \$128,000, dwarfing scanning costs by a factor of 400.
- ❖ More expensive models don't fix the noise problem. Instead, the net result is they shift the costs around, burning far more tokens in exchange for lower labor costs.
- ❖ The non-deterministic nature of AI scanning is problematic. In Contrast's testing, three scanners analyzed the same codebase. They agreed on only 5% of findings. The same scanner, run three times on the same code, reproduced only 17% of its own findings.
- ❖ At portfolio scale, this math becomes untenable fast.

AI is also changing the economics of vulnerability discovery on the attacker side. Automated scanning has always been part of the threat landscape, but AI-assisted tools lower the skill threshold for finding and weaponizing vulnerabilities, enabling broader and faster coverage of the attack surface. As these tools become more capable and more accessible, the rate of new CVE disclosures is likely to accelerate alongside the rate of exploitation.

The result is a vulnerability haystack that grows in both directions at once: more vulnerabilities entering production through AI-assisted development, and more of those vulnerabilities being found and exploited through AI-assisted attack tooling. The prioritization problem becomes harder, not easier, in that environment. The organizations that will manage it most effectively are those that move beyond volume-based triage and build the runtime visibility needed to distinguish the vulnerabilities that matter from the ones that don't.

Takeaway

AI is often not a net positive for the vulnerability landscape, at least not yet. For security leaders, a realistic assessment shows that AI-assisted development is adding risk to the backlog faster than AI-assisted security is removing it, and the mathematical gap between those two trajectories is widening. Investing in AI scanning tools without addressing the noise and inconsistency they produce does not solve the problem; it relocates it, at considerable cost.

The AI question for security leaders is not whether to adopt these tools. It is whether the processes and controls surrounding them are keeping pace with the risk they uncover. The organizations best positioned to navigate this environment are those that are able to reliably sort the signal from the noise, and prioritize the findings that matter most.

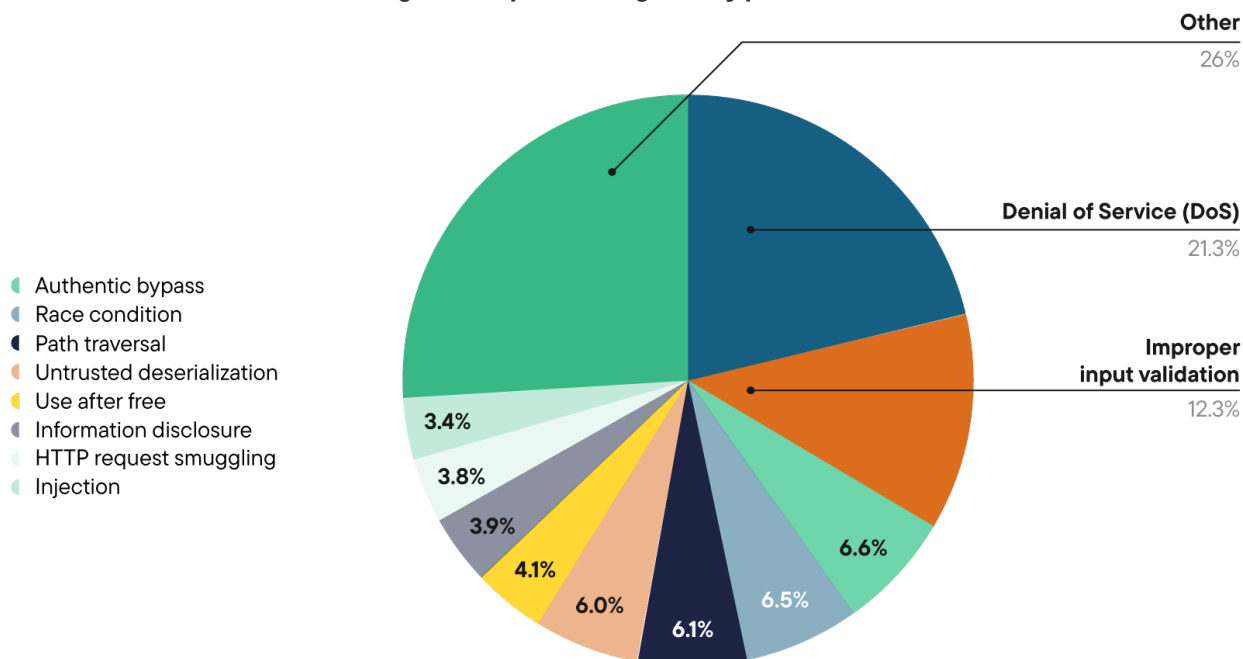
Vulnerability prioritization: Defenders face a prioritization crisis

As more vulnerabilities flood the pipeline, effective prioritization becomes more important than ever. If our remediation resources are limited, then it only makes sense to focus on the risks that are most likely to be targeted successfully by an attacker. Unfortunately, the static metadata most AppSec programs rely upon for prioritization paints a picture that is at best incomplete, and at worst, dangerously misleading.

Application vulnerability categories

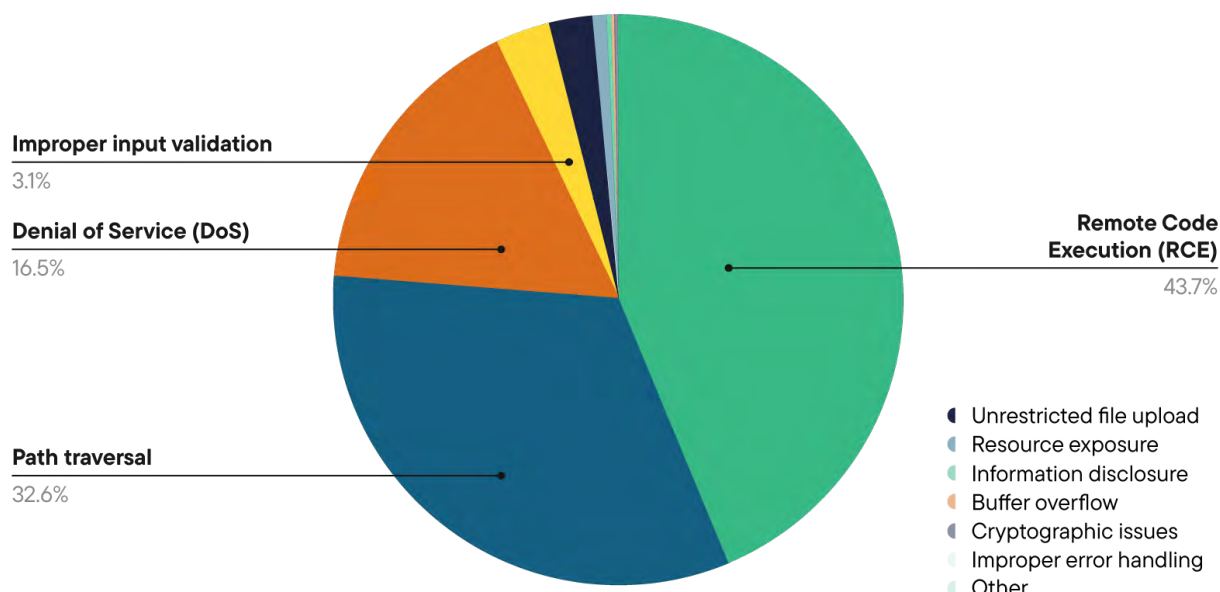
For this analysis, we will focus on vulnerabilities observed in the wild third-party components. This allows us to enrich the data with public context to see a more complete picture. As a baseline, we'll look at the overall categories of observed vulnerabilities.

Figure 8: Top CVE categories by prevalence



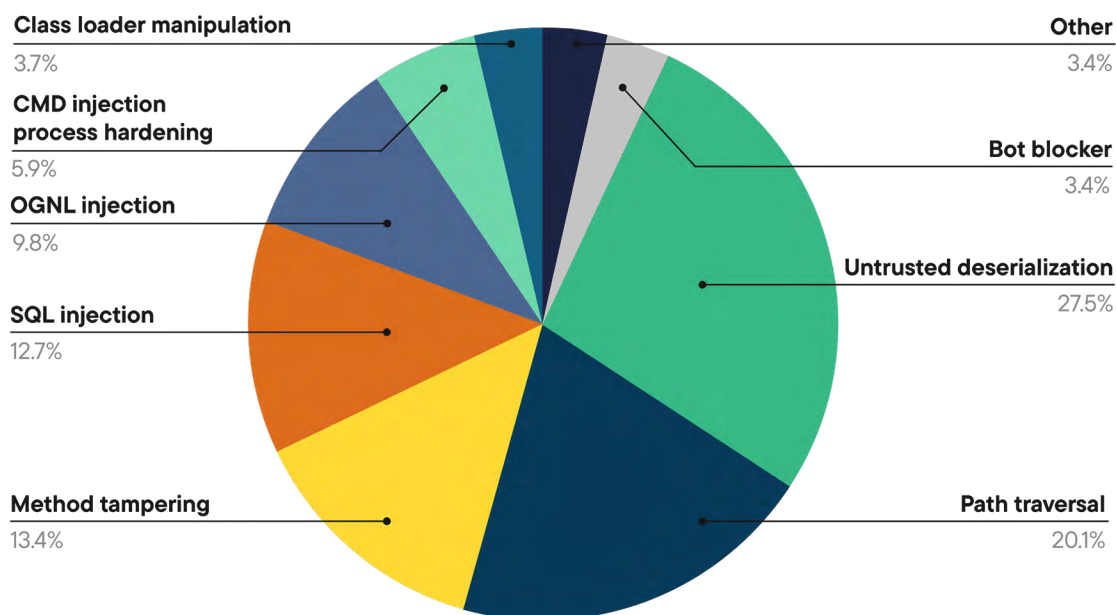
Across all CVEs observed in production environments, Denial of Service (DoS) vulnerabilities account for the largest share by instance count at more than 20%, followed by improper input validation, authentication bypass and race condition. Volume alone would suggest these categories collectively represent the greatest remediation burden organizations face. That over-simplified analysis distorts the shape of actual risk.

Figure 9: Top known–exploited CVE categories by prevalence



If we shift our attention to focus only on known exploited vulnerabilities, the picture shifts considerably. Looking exclusively at CVEs on CISA's Known Exploited Vulnerabilities (KEV) list, Remote code execution dominates at roughly 43% of all observed KEV instances, followed by path traversal at 32% and Denial of Service (DoS) at 16%. DoS shrinks but doesn't disappear, and the categories that bulk up the general CVE population, particularly authentication bypass, race condition and HTTP request smuggling, are largely absent.

Figure 10: Top types of viable application attacks



Neither view, however, reflects the observed techniques that attackers are actively employing against production environments. Revisiting observed viable attacks discussed in viable attacks discussed previously ("The software threat landscape"), we see that untrusted deserialization leads in observed attack volume by a wide margin, followed by path traversal and method tampering. Runtime observations provide critical, ground-truth context that not only helps SecOps teams to identify threats, but also helps AppSec teams to prioritize strategically. It does this by shifting the attention from theoretical risk to actual observed attack activity.

Reachability vs. exploitability

While a particular CVE may have a known exploit, that doesn't mean that every instance of that vulnerability in a particular codebase is a critical finding. Reachability and exploitability provide better contextual analysis.

Reachable vulnerability instances

What it tells us:

"Reachability" tells us that the vulnerability instance is theoretically executable, based on the control flow graph of an application.

How its discovered:

Many modern AppSec scanners incorporate some level of reachability analysis in their findings to help reduce false positives.

What it doesn't tell us:

Reachability ignores important context such as upstream data sanitization, application configuration and compensating controls that mitigate the risk associated with many "reachable" vulnerability instances.

Many "reachable" vulnerability instances introduce no risk, because of these runtime factors.

Exploitable vulnerability instances

What it tells us:

"Exploitability" assesses whether vulnerable code can actually be exploited in real-world situations by answering contextual questions:

- Does the vulnerable library actually run?
- Does the method associated with the CVE actually run?
- Is the application configured in a way that would allow the vulnerability to be exploited?
- Does untrusted data actually reach that method in a way that allows the attacker to exploit it?

How its discovered:

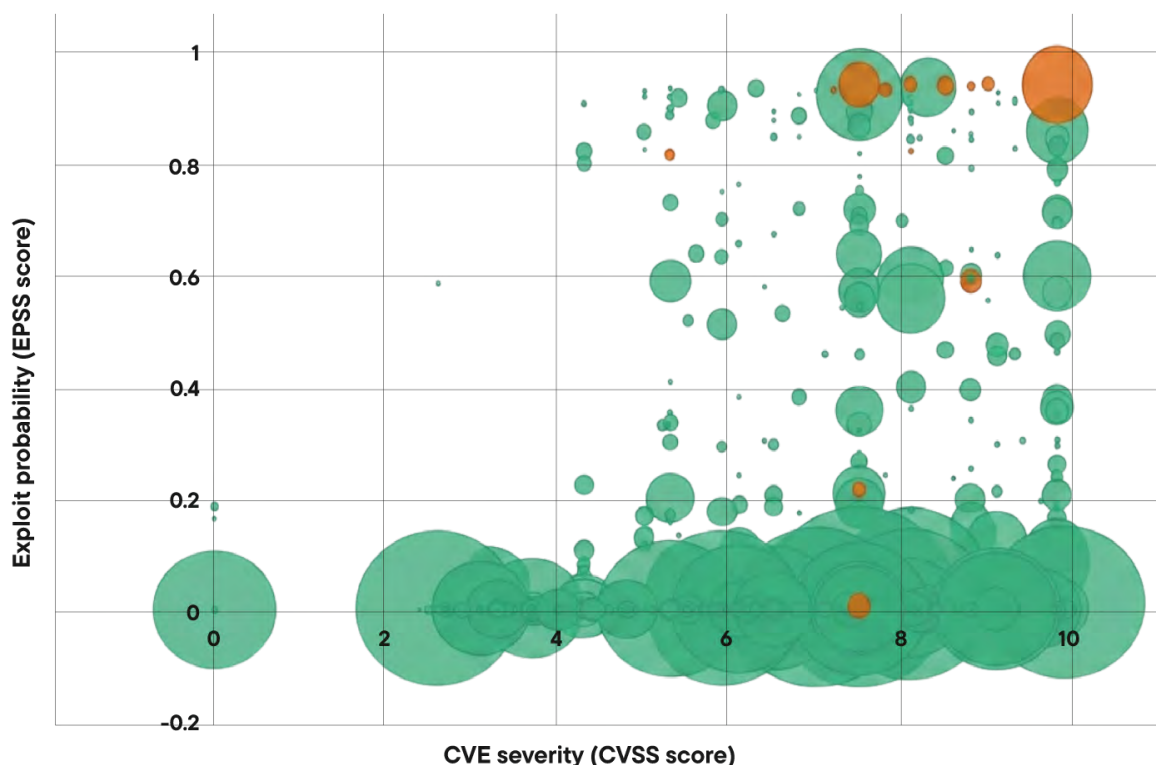
Assessing exploitability requires monitoring applications and APIs at runtime, in order to understand the full and accurate context of the running application.

Prioritization by vulnerability severity and exploitability: Imperfect metrics

The industry's most straightforward answer for prioritization has been Common Vulnerability Scoring System (CVSS) severity scores, which assess the inherent technical impact of a flaw. CVSS was designed to allow organizations to generate tailored scores by incorporating local context such as business impact, compensating controls and other environmental factors. In practice, however, few organizations bother, relying instead on generic CVSS base scores. The result is scores that tend to assume the worst-case scenario, inflating the perceived risk for most vulnerabilities beyond what's real given the facts on the ground.

Another important metric comes from the Exploit Prediction Scoring System (EPSS), which estimates the probability that a given CVE will see active exploitation within the next 30 days. Both metrics carry genuine signal. But when mapped against the CVEs Contrast observed across customer environments in early 2026, the data reveals meaningful gaps in what these scores capture, and where those gaps leave defenders exposed.

Figure 11: Observed application CVE vulnerabilities: severity vs. exploitability



The bubble chart above maps every CVE Contrast observed in customer environments over a 30-day window in early 2026, against two dimensions: CVSS severity on the horizontal axis, and EPSS score on the vertical. Bubble size reflects how frequently each CVE appears across monitored applications. Orange bubbles indicate CVEs on CISA's KEV list.

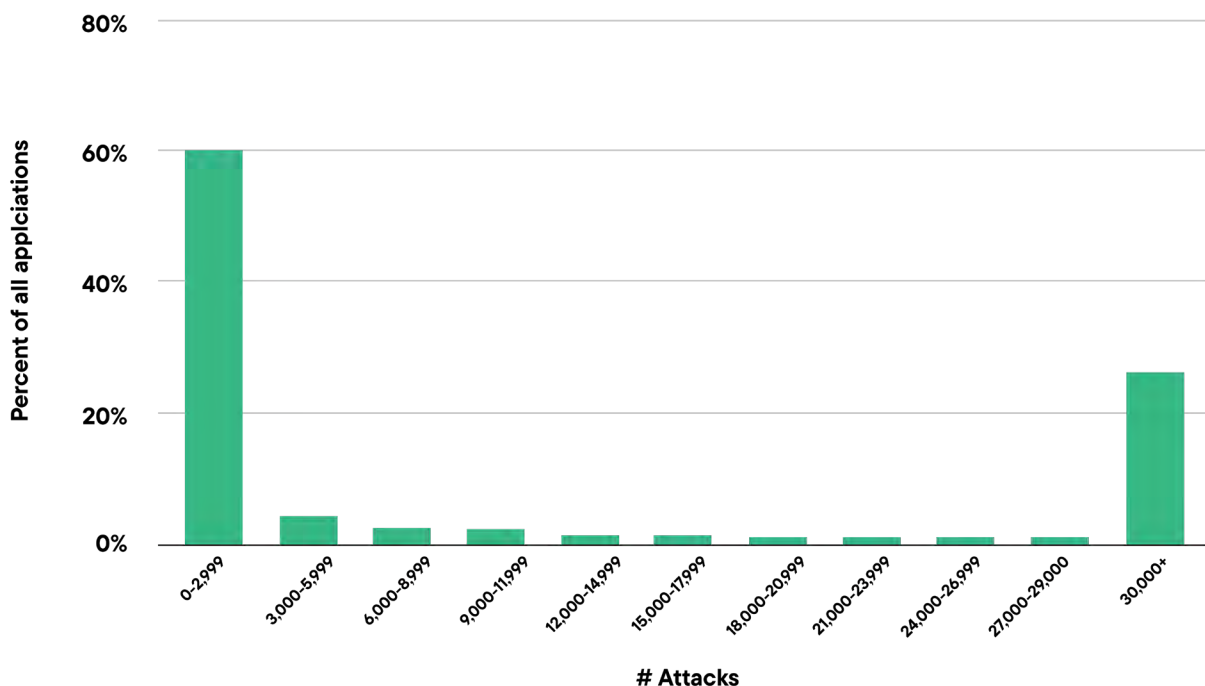
A few patterns stand out immediately:

- The large cluster of bubbles concentrated near the bottom of the chart, spanning a wide range of CVSS scores, represents CVEs that are highly prevalent in customer environments but carry very low exploit probability. Their volume creates noise in any severity-first prioritization approach, pulling remediation attention away from higher-priority issues.
- The orange bubbles tell a different story. Of the 34 CVEs Contrast observed in the data set that appear on the KEV list, 28 (82%) carry an EPSS score of 90% or higher. That relationship strengthens as EPSS scores climb: among CVEs with an EPSS score above 94%, 22 of 26 (85%) are KEV-listed.

High EPSS scores are a strong signal that a vulnerability will attract attacker attention, and that signal overlaps substantially with known active exploitation.

That said, EPSS is not a complete answer. A handful of low-EPSS CVEs appear on the KEV list, including several with meaningful prevalence in Contrast's dataset. Two examples: CVE-2006-1547, a CVSS 7.5 vulnerability with an EPSS score of 22.2%, and CVE-2023-38180, a CVSS 7.5 vulnerability with an EPSS of just 0.88%. Both are confirmed as exploited in the wild despite low predicted probability. Severity and exploit probability are meaningful filters, but neither is sufficient alone.

Figure 12: Attack rate distribution



Severity and exploitability are estimates, but don't tell the full risk story. A vulnerability in an application that's in development or buried deep within an organization brings a great deal less risk than the same vulnerability in an application that's internet-accessible and readily attacked. The distribution of attack volume across applications is sharply uneven: more than 60% of applications see fewer than 3,000 attacks per month, while more than a quarter absorb upward of 30,000. The applications sitting at the high end of that distribution represent concentrated, elevated risk that deserves concentrated attention. Whether they're high-value targets, running widely exploited frameworks or simply more exposed to external traffic, they warrant a different level of scrutiny than the broader application portfolio. Understanding which applications those are, and why, is one of the most high-leverage things a security team can do.

Takeaway

Effective prioritization is the key to tipping AppSec math back in favor of the defender. The core lesson here is that it requires more than generic metadata; it requires organizational context. Categories, severity scores and exploit probability metrics are useful but insufficient in dealing with today's rising tide of vulnerabilities, and none accounts for the specific shape of an organization's attack exposure.

Security leaders who want to make real progress need to combine these signals with additional telemetry that reveals which vulnerabilities are actually reachable and exploitable in their environments. The applications and APIs absorbing the most attack traffic deserve a different level of scrutiny than the rest of the portfolio, and the vulnerabilities most likely to enable persistent access or code execution deserve a different level of urgency than the ones that merely inflate the backlog. This level of precision separates organizations that are meaningfully reducing risk from those that are just closing tickets.

Retooling AppSec for the AI age

AppSec math is broken. The imbalance between attackers and defenders in the application layer is no longer a gap that better tooling at the perimeter can close. AI has changed the math on both sides of the equation, and the change is not symmetric.

On the attacker's side, AI is compressing every step of the attack lifecycle. Vulnerability discovery that used to require deep specialist expertise can now be largely automated. Exploit code that took weeks to weaponize is being produced in hours. Reconnaissance, payload crafting, and evasion are all being industrialized through models that work tirelessly, cheaply and at scale.

On the defensive side, AI is also accelerating development, producing more code, more dependencies, more APIs and more attack surface, often shipped faster than security teams can review it. AI coding assistants generate working software, but they also generate vulnerabilities at a rate that traditional AppSec workflows were never designed to absorb. AI scanners are showing promise at discovering vulnerabilities, but lack context for effective prioritization. This creates a work imbalance that security teams will never recover from.

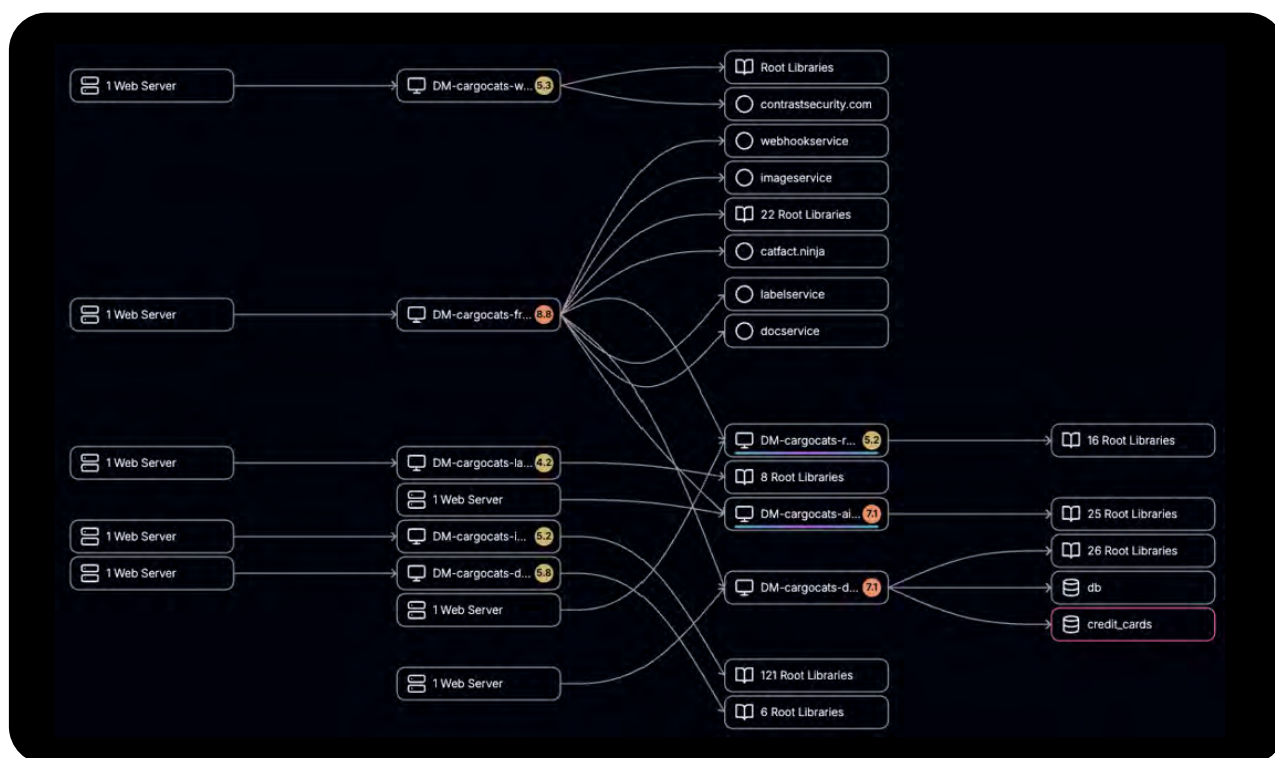
This is the structural problem defenders face heading into 2026 and beyond. The traditional playbook of find, prioritize and remediate remains necessary. It is not, by itself, sufficient. No remediation cadence can match the velocity at which AI-assisted attackers identify and exploit weaknesses, and no review process can keep pace with AI-assisted code generation without slowing the business to a crawl.

What changes the game is shifting defensive posture inside the application itself. Runtime visibility into how applications and APIs actually behave under attack, combined with the ability to block exploitation attempts in real time, neutralizes the velocity advantage attackers have built. Vulnerabilities that have not yet been patched, that have not yet been discovered, or that have been introduced by AI-generated code can all be defended at the moment of exploitation, rather than waiting for the next patch cycle.

Contrast was built for the age of AI

The [Contrast runtime security platform](#) provides real-time, always-on security inside your applications and APIs, allowing AppSec and SecOps teams to observe and prioritize vulnerabilities while also detecting and defending against application-layer attacks.

Figure 13: The Contrast Graph

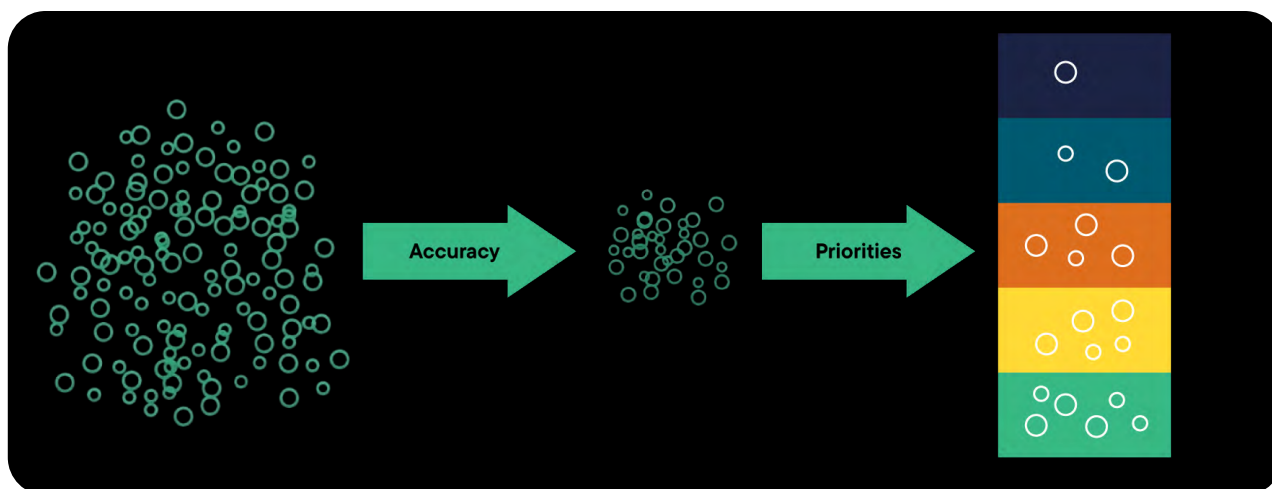


Powered by the Contrast Graph

The [Contrast Graph](#) lies at the core of the Contrast runtime security platform, powering advanced capabilities including optional Agentic AI workflows that help teams respond faster and fix smarter. The Graph builds a real-time digital twin of an organization's application and API environment, mapping live attack paths; correlating runtime behavior; and exposing how vulnerabilities, threats and assets are connected.

Contrast Score: dynamic, contextual risk

The Contrast Graph uses production risk factors like asset criticality, runtime exploitability, threat intelligence and active attacks to dynamically update risk scores, ensuring focus is always on the findings that truly matter.



Traditional findings are bloated with false positives and inflated risk scores

Runtime technical context drives accuracy by eliminating false positives

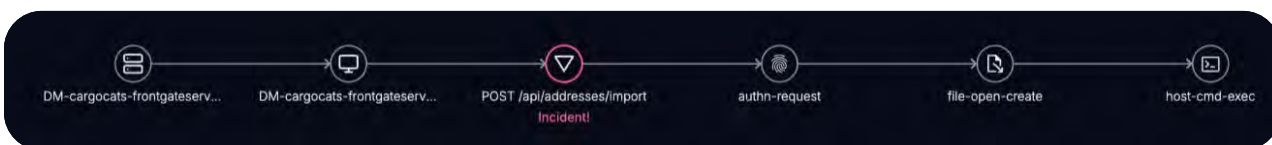
Examples:

- Function inputs are sanitized before processing
- Runtime blocking or other security controls prevent exploitation
- Vulnerable function is never invoked

Runtime threat, architecture, and business context drives priorities based on actual risk

Examples:

- Active exploit activity has been observed in the wild
- Finding exists production environment
- Function has write access to business-critical cloud repository
- Function is reachable without authentication



Proactive application protection

Contrast not only uncovers vulnerabilities, but also provides high-fidelity detection of runtime behavioral anomalies that enables precise, optional in-application blocking of attacks. Contrast's runtime security platform can be configured to intervene exactly where needed, halting the specific malicious operation or request within the runtime, offering effective protection with minimal disruption.

Reimagine security for applications

Contrast's runtime security platform provides the tools that power effective AppSec in the age of AI. Experience how Contrast can help modernize your security stack with a guided walkthrough or a hands-on demonstration.

Try Contrast

¹[Claude Mythos: Get your AppSec game on](#)

²[AI-Led Remediation Crisis Prompts HackerOne to Pause Bug Bounties](#)

³[Security Bug Bounty Program Paused Due to Loss of Funding](#)

⁴[VulnCheck State of Exploitation 2026](#)

Contrast Security is the world's leader in Runtime Application Security, embedding code analysis and attack prevention directly into software. Contrast's patented security instrumentation enables powerful Application Security Testing and Application Detection and Response, allowing developers, AppSec teams and SecOps teams to better protect and defend their applications against the ever-evolving threat landscape.

© 2026 Contrast Security, Inc.

contrastsecurity.com

6800 Koll Center Parkway
Ste 235
Pleasanton, CA 94566
Phone: 888.371.1333

